SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Scenario Based Assignment
Weightage:	20% of CIA	**Duration / Marks:**	60 mins

CO2 Statement: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Student Name:	**B. Muni Chandini**	**Reg. No.:**	**192524283**
Section / Batch:	_AI&DS	**Date:**	22/10/2026

Code of Conduct

I, B. Muni Chandini (Reg. No. 192524283), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:*B.Muni Chandini*

Instructions

• Read each scenario carefully before answering (Off-Class Assessment)

• Answer all five scenario-based questions. Each question carries and is designed to be answered in about 12 minutes.

• Identify the key issues in each scenario, apply the relevant AI technique, and justify every decision with clear reasoning.

• Questions are drawn from Informed Search (Greedy, A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).

• Marks will be awarded using the Scenario Based Assignment rubric: understanding of scenario, identification of key issues, application of concepts, decision making and clarity of response.

• Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions	Q1
2. Local Search and Online Search	Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms), online search agent	Q2, Q3
3. Constraint Satisfaction Problem	Variables, domains, constraints, backtracking search, constraint propagation (forward checking)	Q4
4. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	Q5

Rubrics for Scenario Based Assignment

Criteria	Weightage	Level 4 – Exemplary (100%)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Marking Sheet

Q. No.	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total
1						
2						
3						
4						
5						

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions)*

Scenario Summary: AI-powered drones deliver emergency medicines in a flood-affected region. The environment is highly dynamic with blocked roads and variable weather-dependent costs. The drone has a straight-line distance heuristic, partial real-time updates on blocked paths, and must minimize delay and risk.

(a) Behaviour of Greedy Best-First Search – Strengths and Weaknesses under Uncertainty

Greedy Best-First Search expands the node that appears closest to the goal according to the heuristic function h(n) alone. In this flood-zone scenario, the drone would always choose the neighbouring location with the smallest straight-line distance to the destination hospital or clinic, ignoring the actual path cost so far.

Strengths:

- Extremely fast decision-making – only the heuristic is evaluated, making it suitable when the drone must react within seconds.
- Low computational overhead and memory usage, which is advantageous for onboard drone processors with limited resources.
- Can quickly produce a plausible route when a good heuristic (straight-line distance) is available.

Weaknesses under uncertainty:

- Completely ignores the actual cost (fuel, time, risk) already incurred. A path that looks short may traverse flooded or high-wind zones, leading to higher real cost or failure.
- Not optimal and not even complete in the presence of obstacles; it can be trapped in dead-ends or loops when blocked paths appear.
- Highly sensitive to heuristic inaccuracy. In a flood zone, straight-line distance can be misleading because the terrain between two points may be impassable.

(b) How A* Handles the Situation – Balancing Cost and Heuristic

A* search evaluates nodes using the evaluation function f(n) = g(n) + h(n), where g(n) is the actual path cost from the start (accumulated travel time, energy, risk) and h(n) is the estimated remaining cost (straight-line distance converted into an expected time/cost).

In the drone scenario, A* maintains a priority queue ordered by f(n). At each step it expands the node with the lowest f-value. This naturally balances the desire to stay close to the goal (h) against the real cost already paid (g). When weather increases the cost of a segment, that increase is reflected in g(n) for all subsequent nodes, so A* will prefer safer or cheaper alternatives even if they appear slightly longer on the map.

Provided the heuristic is admissible (never overestimates the true remaining cost) and consistent, A* is guaranteed to find an optimal path with respect to the cost model that is currently known. Partial real-time updates can be incorporated by re-planning from the current position whenever a significant blockage is detected.

(c) Impact of Heuristic Accuracy on Both Algorithms

Heuristic accuracy is critical for both algorithms but affects them differently.

Greedy Best-First Search: Performance depends almost entirely on h(n). An accurate heuristic yields fast, near-optimal routes; an inaccurate one (e.g., straight-line distance that ignores large

flooded areas) can lead the drone into costly detours or dead-ends. Because g(n) is ignored, even small heuristic errors are amplified.

A* Search: As long as h(n) remains admissible, optimality is preserved. A tighter (more accurate) admissible heuristic reduces the number of nodes expanded, improving speed. If the heuristic becomes inconsistent or overestimates because of outdated map data, A* may lose optimality or expand more nodes. In practice, a carefully designed cost-aware heuristic that incorporates expected weather risk still keeps A* efficient while guaranteeing a safe route.

(d) Effect of Dynamic Changes (Blocked Paths) on Search Performance

When a path suddenly becomes blocked, any search tree that already contains that edge becomes invalid. Greedy search may continue toward the blocked region until it reaches a dead-end, then backtrack inefficiently. A* can detect the blockage, increase the cost of the affected edge to infinity, and re-open nodes whose f-values are affected. Incremental or lifelong planning variants (D*, D* Lite, LPA*) are especially useful: they reuse previous search information and only re-expand the portions of the graph that changed, enabling the drone to replan in near real time.

Frequent map updates increase the computational load. Without incremental techniques, both algorithms may have to restart from scratch, which is costly for an airborne platform. Therefore, the search must be designed to support rapid re-planning and to keep a safety buffer around the current trajectory.

(e) Recommendation and Justification

Recommended algorithm: A* (or an incremental variant such as D* Lite) with an admissible, weather-aware heuristic.

Justification:

- Optimality & Safety: A* guarantees the lowest-cost path with respect to the current cost model, reducing the risk of the drone running out of energy or entering dangerous weather.
- Real-time constraints: With a good heuristic and incremental re-planning, A* can produce new routes fast enough for a drone that must react to sudden blockages.
- Handling uncertainty: By incorporating real path costs and allowing dynamic updates, A* is far more robust than pure greedy search.
- Practical compromise: If onboard compute is extremely limited, a hybrid approach can be used – greedy search for the immediate horizon combined with periodic A* re-planning for the global route.

In life-critical medicine delivery, the extra computational cost of A* is justified by the gain in reliability and safety. Therefore A* (with incremental updates) is the most suitable choice.

Question 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms)

Scenario Summary: An AI system must continuously adjust traffic-signal timings across hundreds of intersections to minimise total waiting time, fuel consumption and congestion. The search space is enormous, traffic patterns change dynamically, and the system risks becoming stuck in locally optimal solutions.

(a) Formulation as an Optimization Problem & Why Traditional Search is Inefficient

The problem can be formulated as a continuous or high-dimensional discrete optimization task. Let the decision variables be the green-time durations (and possibly offsets and cycle lengths) for every signal phase at every intersection. The objective function is a weighted sum:

$$f(x) = w1 \cdot TotalWaitingTime(x) + w2 \cdot FuelConsumption(x) + w3 \cdot CongestionIndex(x)$$

where x is the vector of all signal parameters. Constraints include minimum and maximum green times, pedestrian clearance intervals, and coordination requirements between adjacent intersections.

Traditional systematic search (breadth-first, depth-first, A*, etc.) is inefficient because the state space grows exponentially with the number of intersections and possible timing combinations. Evaluating every feasible assignment is computationally impossible in real time. Moreover, the objective landscape is non-convex and changes continuously with traffic flow, rendering exhaustive or even informed tree-search impractical.

(b) Hill Climbing – Working Principle and Limitations

Hill Climbing starts with a current signal-timing configuration and iteratively moves to a neighbouring configuration that improves the objective function (lower waiting time, fuel use, etc.). Neighbours can be generated by slightly increasing or decreasing a green time, shifting an offset, or swapping phase sequences. The algorithm accepts the best improving neighbour and repeats until no further improvement is possible.

Limitations in this scenario:

- Gets stuck at local maxima – a timing plan that is good for the current traffic pattern but far from the global optimum.
- Cannot escape plateaus where many neighbouring plans yield almost identical objective values.
- Sensitive to the initial configuration; a poor starting plan may never reach a high-quality solution.

(c) Local Maxima, Plateaus and Ridges – Real-World Interpretation

Local maxima: A set of signal timings that minimises congestion in one corridor while causing severe queues on a perpendicular arterial. Any small change worsens the measured objective, so the algorithm stops, even though a coordinated city-wide plan would be far better.

Plateaus: During off-peak hours many timing adjustments produce nearly identical average delays. The search wanders randomly without clear improvement, wasting computation and failing to adapt when traffic begins to build.

Ridges: A narrow “ridge” of good solutions corresponding to a progressive green-wave along a major road. Moving perpendicular to the ridge (changing an unrelated intersection) immediately degrades performance, making it hard for simple hill climbing to follow the ridge toward the global optimum.

(d) How Simulated Annealing or Genetic Algorithms Overcome These Limitations

Simulated Annealing (SA): SA occasionally accepts worse solutions with a probability that decreases over time according to a temperature schedule. Early in the search the system can jump out of local maxima and explore distant regions of the timing space. As temperature cools, the algorithm becomes more greedy and refines a good solution. This stochastic escape mechanism directly addresses the local-maxima problem of hill climbing.

Genetic Algorithms (GA): A population of candidate timing plans is maintained. Selection, crossover (combining green times from two parents) and mutation (random small changes) generate new offspring. Because multiple solutions are explored in parallel and genetic operators can recombine useful partial plans, GAs are less likely to be trapped by local optima and can discover coordinated strategies that simple local search would miss. They also adapt naturally when traffic patterns change by continuously evolving the population.

(e) Recommended Approach and Justification

Recommended approach: A hybrid of Genetic Algorithms with occasional Simulated-Annealing-style local refinement, running continuously in a rolling-horizon fashion.

Justification based on scalability and adaptability:

- Scalability: Population-based GAs can be parallelised across many CPU cores or edge servers; each individual evaluation can be a fast microscopic traffic simulation or a learned surrogate model.
- Adaptability: The evolutionary process never stops; when traffic demand shifts, the fitness landscape changes and the population automatically migrates toward new good regions.
- Robustness: Maintaining diversity in the population prevents premature convergence and provides a set of alternative plans that can be switched to if an unexpected incident occurs.
- Practical performance: Modern cities already use evolutionary and meta-heuristic methods for signal optimisation with proven reductions in delay and emissions.

Thus a continuously running genetic algorithm, optionally hybridised with simulated annealing for local polishing, offers the best combination of global search power, scalability and real-time adaptability for city-wide traffic-signal optimisation.

Question 3: Autonomous Rover Exploring Mars

Topic: Online search agent

Scenario Summary: An autonomous rover must explore unknown Martian terrain, collect samples, avoid hazards (craters, rocks), operate with limited sensing, and make decisions without a complete map. Knowledge is updated as the rover moves, turning the task into a real-time decision-making problem.

(a) Why an Online Search Agent is Required Instead of Offline Search

Offline search assumes a complete and accurate model of the environment is available before any action is taken. The agent can compute an entire path from start to goal and then execute it open-loop. On Mars this assumption is false: the terrain map is incomplete, sensor range is limited, and new hazards may appear (or previously unseen safe paths may be discovered). Consequently the rover cannot plan a full trajectory in advance.

An online search agent interleaves planning and execution. At each step it decides the next action based on the information gathered so far, executes that action, observes the new percepts, updates its internal model, and repeats. This closed-loop behaviour is mandatory when the environment is only partially known and must be explored while the mission is under way.

(b) Challenges of Partial Observability and Dynamic Environments

Partial observability:

- Sensors have limited range and may be occluded by rocks or dust; the true state of distant terrain remains unknown.
- The rover must maintain a belief over possible maps and choose actions that both progress toward scientific goals and reduce uncertainty (information-gathering actions).
- Sensor noise and localisation error further complicate the mapping process.

Dynamic environments:

- Although Mars is largely static, dust storms, temperature-induced rock falls or the rover's own tracks can alter traversability.
- Communication delays with Earth mean that any ground-in-the-loop correction arrives too late; the rover must react autonomously to unexpected events.

(c) How the Rover Builds and Updates its Internal Model

The rover maintains an occupancy-grid or topological map that stores, for each cell or node, an estimate of traversability, hazard probability and scientific value. As new sensor data arrive:

1. Localisation (visual odometry / SLAM) determines the rover's current pose relative to the map.
2. Newly observed cells are marked free, occupied or unknown; probabilistic filters (e.g., Bayesian updates) refine occupancy probabilities.
3. Cost maps are regenerated: high cost for rocky or steep terrain, low cost for smooth safe ground.
4. The online planner (e.g., D* Lite, RRT with rewiring, or a learning-based policy) recomputes a path from the current pose toward the next sample site, treating unknown space with a cautious default cost.

This continuous sense–map–plan–act loop keeps the internal model consistent with reality and allows the rover to improve its knowledge while progressing on the mission.

(d) Comparison of Online Search with Uninformed and Informed Search Strategies

Uninformed search (BFS, DFS, Uniform-Cost) assumes a fully known graph and expands nodes systematically. It cannot be applied directly on Mars because the graph is incomplete; expanding an unknown edge would require the rover to physically travel there first.

Informed search (Greedy, A*) also presupposes a known state space and an admissible heuristic defined over that space. While a heuristic based on Euclidean distance to a sample site can still be used, the underlying graph must be discovered on the fly. Pure offline A* would plan through unknown cells and then fail when those cells turn out to be impassable.

Online search algorithms (LRTA*, Real-Time A*, D* family, etc.) explicitly handle the interleaving of exploration and exploitation. They guarantee progress toward the goal while never requiring knowledge of the entire map in advance. They may sacrifice global optimality for real-time responsiveness and safety, which is an acceptable trade-off for a planetary rover.

(e) Most Suitable Approach – Justification

The most suitable approach is an online search agent that combines incremental path planning (D* Lite or similar) with a probabilistic occupancy map and a safety-aware cost function.

Justification:

- Uncertainty: The algorithm never assumes knowledge it does not possess; unknown cells are treated cautiously.
- Adaptability: When new sensor data arrive, only the affected portion of the search tree is repaired, enabling rapid re-planning.
- Safety: Cost functions can encode risk (slope, rock density, energy margin), so the rover preferentially chooses safer corridors even if they are longer.
- Mission success: By continuously updating its model and re-planning, the rover maximises the probability of reaching scientifically valuable sites while remaining within energy and communication constraints.

Hence an online, incremental, safety-aware search agent is the only practical solution for autonomous Martian exploration under partial observability.

Question 4: University Exam Timetable Generation

Topic: Constraint satisfaction problem (backtracking search, constraint propagation, forward checking)

Scenario Summary: An AI system must automatically generate exam timetables satisfying constraints such as no overlapping exams for any student, limited exam halls, precedence among subjects, faculty availability, and special accommodations. Complexity grows with the number of courses and students.

(a) Formulation as a Constraint Satisfaction Problem (CSP)

Variables: For each exam E_i (i = 1 … n) we introduce a variable X_i whose value is a pair (time-slot, hall). Alternatively we may use two variables per exam: Time_i and Hall_i.

Domains: Dom(Time_i) = {T1, T2, …, Tm} – the set of available examination periods. Dom(Hall_i) = {H1, H2, …, Hk} – the set of rooms, possibly further restricted by capacity and special-accommodation requirements.

Constraints:

- Student conflict: If two exams share at least one common student, then Time_i ≠ Time_j.
- Hall capacity: For any time slot t the set of exams assigned to the same hall must not exceed the hall's seating capacity.
- Precedence: For certain subject pairs, Time_i must precede Time_j (hard ordering constraint).
- Faculty availability: An exam can be scheduled only in a slot when its invigilating faculty is free.
- Special accommodations: Certain students require specific halls or extended time; these become unary or binary constraints on the corresponding variables.

A complete assignment of values to all variables that satisfies every constraint constitutes a feasible timetable.

(b) How Backtracking Search Works in this Scenario

Backtracking search builds a partial assignment systematically. It selects an unassigned exam variable (using a variable-ordering heuristic such as Minimum Remaining Values), then tries values from its domain (ordered by a value-ordering heuristic such as Least Constraining Value). After each assignment it checks consistency with already assigned variables. If a conflict is detected the algorithm backtracks, undoing the most recent assignment and trying the next value. When a variable has no remaining legal values the search backtracks further. The process continues until a complete consistent assignment is found or the search tree is exhausted.

In the timetable domain this means the system places one exam after another into a free slot/hall combination, immediately verifying that no student is double-booked, that the hall is large enough, that faculty are available, etc. As soon as a violation appears the current partial timetable is abandoned and an alternative placement is attempted.

(c) Constraint Propagation (Forward Checking) – Efficiency Improvement

Forward checking is a limited form of constraint propagation performed after every assignment. When an exam is placed in a particular time slot, forward checking immediately removes that time slot from the domains of all other exams that share students with the newly assigned exam. Similarly, the chosen hall's remaining capacity is reduced and any hall that can no longer accommodate a still-unassigned exam is pruned from that exam's domain.

By detecting domain wipe-outs early, forward checking prunes large portions of the search tree before they are explored. Combined with arc-consistency algorithms (AC-3) that enforce consistency among all pairs of variables, the search can solve realistically sized university instances that would be

intractable for plain chronological backtracking. The net effect is a dramatic reduction in the number of partial assignments that must be examined, making automated timetable generation practical.

(d) Real-World Challenges and Best Strategy for Scalability

Real-world challenges include:

- Very large numbers of courses and students produce an enormous combinatorial space.
- Soft constraints (preferred slots, balanced workload for invigilators) must be optimised in addition to hard constraints.
- Last-minute changes (faculty illness, room unavailability) require rapid re-scheduling.
- Fairness and stakeholder acceptance demand transparent, explainable solutions.

Best strategy for scalability:

A hybrid CSP approach that combines backtracking with aggressive constraint propagation (forward checking + AC-3 or even stronger consistency), intelligent variable/value ordering, and, when pure CSP search becomes too slow, a meta-heuristic layer (local search or genetic algorithm) that repairs an almost-feasible timetable. Decomposition techniques that solve departmental sub-problems first and then integrate them also improve scalability. Modern solvers further employ nogood learning and parallel search, enabling universities to generate high-quality, constraint-satisfying examination timetables within acceptable time limits even for thousands of students and hundreds of courses.

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing – adversarial search (minimax, alpha-beta pruning, evaluation functions)

Scenario Summary: A gaming company develops an AI opponent for a real-time strategy (RTS) game. The AI must compete against human players, decide within strict time limits, and evaluate complex game states with many possible moves. The decision tree is extremely large, making computation expensive.

(a) How the Minimax Algorithm Models this Problem

Minimax models the game as a two-player zero-sum adversarial search problem. The AI (MAX) tries to maximise its chance of winning while the opponent (MIN) tries to minimise that chance. The game tree is expanded to a fixed depth or until terminal positions are reached. At leaf nodes an evaluation function returns a numeric score estimating the desirability of the position for MAX. Values are backed up: at MAX nodes the highest child value is chosen; at MIN nodes the lowest child value is chosen. The move that leads to the highest backed-up value at the root is selected for execution.

In an RTS setting the "moves" correspond to high-level strategic actions (attack a base, expand economy, research a technology, etc.) rather than every individual unit command, so that the branching factor remains manageable.

(b) Computational Challenges of Large Game Trees

RTS games possess enormous state and action spaces. A single decision point may have dozens or hundreds of legal high-level actions; each subsequent ply multiplies the number of nodes. Even a shallow search of depth 4–6 can generate millions of positions. Evaluating each position requires simulating combat outcomes, resource flows and unit positions, which is itself expensive. Strict real-time constraints (often only a few tens of milliseconds per decision) make exhaustive minimax infeasible. Additional complications include simultaneous moves, imperfect information (fog of war) and continuous time, all of which further enlarge the effective search space.

(c) How Alpha-Beta Pruning Improves Efficiency

Alpha-beta pruning maintains two bounds: α (the best value MAX can already guarantee) and β (the best value MIN can already guarantee). While exploring a node, if a move is found that is worse for the player to move than a previously examined alternative, the remaining sibling moves can be pruned because the opponent will never allow the search to reach them. In the best case (perfect move ordering) alpha-beta reduces the effective branching factor from b to approximately $\sqrt{b}$, allowing the same computational budget to reach roughly twice the depth. Even with average ordering the savings are substantial. In practice, good move ordering (e.g., trying captures or previously successful strategies first) maximises the number of cut-offs and is essential for real-time performance.

(d) Design of an Evaluation Function and Justification of Factors

A practical evaluation function for an RTS AI can be a weighted linear combination of material, positional and strategic features:

$$Eval = w1 \cdot MilitaryStrength + w2 \cdot EconomicPower + w3 \cdot TerritorialControl + w4 \cdot TechAdvantage + w5 \cdot WorkerSafety - w6 \cdot ThreatLevel$$

Justification of factors:

- MilitaryStrength (army size, unit quality, hit points) – directly determines combat outcome.
- EconomicPower (resources, production buildings, worker count) – underpins long-term sustainability.

- TerritorialControl (map area, strategic locations held) – provides vision and expansion opportunities.
- TechAdvantage – unlocks stronger units and abilities.
- ThreatLevel – penalises exposed workers or bases that can be attacked immediately, encouraging defensive play when necessary.

Weights can be tuned by hand, by supervised learning from expert games, or by self-play reinforcement learning so that the evaluation correlates well with actual win probability.

(e) Strategy for Balancing Optimality and Real-Time Performance

Recommended strategy: Iterative-deepening alpha-beta (or Monte-Carlo Tree Search) with a strong evaluation function, move ordering, transposition tables, and a hard time cut-off.

Concrete measures:

- Search only to a depth that can be completed within the allotted time; return the best move found so far when the timer expires.
- Use domain knowledge to prune obviously inferior actions (action abstraction).
- Maintain a transposition table so that identical positions reached via different move orders are evaluated only once.
- Combine minimax for short-term tactical fights with a higher-level strategic planner that sets goals (build order, attack timing).
- Optionally employ Monte-Carlo Tree Search, which scales gracefully with available time and naturally handles large branching factors.

By accepting bounded sub-optimality in exchange for guaranteed responsiveness, the AI remains competitive against human players while always returning a decision within the real-time limit. This pragmatic balance between optimality and speed is essential for a commercial real-time strategy game AI.